

Base de datos NoSQL

Documental



Agenda

- Presentación de MongoDB
- Estructura
- Instalación
- Creación y selección de bases de datos.
- CRUD de Colecciones y Documentos
- Documentos embebidos



MongoDB – ¿Que es?

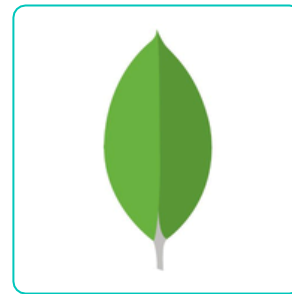
- MongoDB es una base de datos NoSQL de código abierto (escrito en C++)
 - <https://www.mongodb.com/es>
- Posee almacenamiento documental
- Alta disponibilidad
- Fácil escalabilidad



MongoDB – Adecuada para...

- Cuando la aplicación va a tener un crecimiento rápido
- Cuando la estructura de los datos no es siempre la misma
- Cuando la información es dinámica
- Cuando hay muchos usuarios accediendo al mismo tiempo





- Mongo Server
 - Base de datos
 - Colecciones
 - Documentos
 - {campo1: valor1, campo2: valor2, ..., campoN: valorN}

MongoDB - Estructura



MongoDB – Estructura – Colecciones y Documentos

- Los **documentos** son una abstracción de un objeto de la vida real.
- Son un conjunto de pares clave: valor.
- Por ejemplo, una persona:
 - {DNI: 123456, apellido: "apellido 1", nombre: "nombre 1", CUIT: '271234565'}
- **Colecciones:** conjunto de documentos que representan una misma entidad. Todas las personas, todos los productos, todos los usuarios, etc.



- Descargar la versión según el OS:
 - <https://www.mongodb.com/download-center/community>
 - Windows: mongodb-win32-x86_64-2008plus-ssl-4.0.9-signed.msi
 - En la raíz del disco en donde se instaló MongoDB hay que crear un directorio llamado "data" y dentro de este un directorio llamado "db".
- **Documentación:** <https://docs.mongodb.com/manual/installation/>
 - [MongoDB Community Edition Installation Tutorials](#)
 - [Install MongoDB Community Edition on Ubuntu](#)
 - [Install MongoDB Community Edition on Windows](#)
 - [Install MongoDB Community Edition on macOS](#)
 - [Install MongoDB Community Edition on Debian](#)

MongoDB - Instalación

MongoDB - Iniciando



Levantar
servidor
MongoDB:
mongod.exe

el
de



Abrir un **cliente**
de MongoDB:
mongo.exe

MongoDB – Clientes

Studio 3T Community Edition es una herramienta gratuita y no comercial para la gestión de bases de datos, diseñada para ayudarte a iniciar tu experiencia con **MongoDB**.



<https://robomongo.org/>

MongoDB – Iniciando



- **Crear** una base de datos
 - **use** nombre de la base de datos (**por ejemplo:** use actividad)
 - El comando “**use**” selecciona la base de datos con la que se quiere trabajar, en caso de no existir la crea.
- **Ver** la base seleccionada para trabajar:
 - **db**
- **Ver** el listado de bases de datos:
 - **show dbs**
- **Eliminar** una base de datos:
 - Una vez situados en la base de datos a eliminar: **db.dropDatabase()**

MongoDB – Iniciando

- **Crear** una colección de forma explícita:
 - **db.createCollection**("nombre de la colección")
- **Crear** una colección de forma implícita:
 - **db.personas.insert**({DNI: 123456, apellido: "Ape 1", nombre: "Nom 1"})
- **Ver** el listado de colecciones:
 - **show collections**
- **Eliminar** colecciones:
 - **db.nombre_coleccion.drop()**



MongoDB – Iniciando



- **Insertar** documentos en colecciones:
 - `db.nombre_coleccion.insert({documento})`
 - `db.nombre_coleccion.insert([ {doc1}, {doc2}, ..., {docN} ])`
 - Arreglo de documentos.
 - `db.nombre_coleccion.save(documento)`
- **Visualizar** documentos de las colecciones:
 - `db.nombre_coleccion.find()`
 - `db.nombre_coleccion.find().pretty()`
 - `db.nombre_coleccion.findOne()`

MongoDB – Iniciando

○ Edición de documentos:

- **db.nombre_coleccion.update**({condición}, { \$set: {nuevos valores} }, [{multi: true}])
 - **db.personas.update**({ 'DNI' : 123456 }, { \$set : { 'nombre' : 'Lucho' } })
 - El tercer parámetro ({multi: true}) es opcional.
 - Actualiza todos los documentos que cumplen con la condición.
- **db.nombre_coleccion.save**(documento)

○ Eliminar documentos:

- **db.nombre_coleccion.deleteOne**({condición})
- **db.nombre_coleccion.remove**({condición})



MongoDB – Iniciando



○ Consultas:

- **db.personas.find**({ 'CUIT' : '271234565' })
- **db.personas.find**({ 'edad' : { **\$lt**: 30 } }) (menor que)
 - **\$lte** (menor o igual que)
 - **\$gt** (mayor que)
 - **\$gte** (mayor o igual que)
 - **\$ne** (negación)
 - { {campo: valor, campo: valor} } (**AND**)
 - { **\$or** : [{campo:valor}, {campo:valor}] } (**OR**)
 - {campo: valor, **\$or** [{ {campo:valor}, {campo:valor}]] } (**AND + OR**)
 - { campo: { **\$in**: ["A", "D"] } }) (**IN**)

MongoDB – Iniciando



- Algunos ejemplos del comando **find()**:
 - **db.socio.find**({ estado: "A", calificacion: { \$lt: 5 } })
 - SELECT * FROM socio WHERE ((estado = "A") AND (calificacion < 5))
 - **db.socio.find**({ \$or: [{ estado: "A" }, { calificacion: { \$lt: 5 } }] })
 - SELECT * FROM socio WHERE ((estado = "A") OR (calificacion < 5))
 - **db.socio.find**({
estado: "A",
\$or: [{ calificacion: { \$lt: 5 } }, { apellido: /^p/ }] })
 - SELECT * FROM socio WHERE ((estado = "A") AND ((calificacion < 5) OR (apellido LIKE "p%")))

MongoDB – Iniciando



- **db.coleccion.find().limit**(un valor de límite)
- **db.coleccion.find().sort**({criterios de orden})
 - db.personas.find().sort({ apellido: 1}) obtiene las personas ordenadas por apellido de forma ascendente.
- **db.coleccion.find().count()**
- **db.coleccion.find().size()**

MongoDB – Iniciando

○ Cursores

- Nombre del cursor = resultado de una consulta.
 - **var** mi_cursor = db.docentes.find({categoria: 5})
- El cursor posee métodos:
 - forEach
 - mi_cursor.forEach(function(d) { print(d.campo)})
 - mi_cursor.count()
 - mi_cursor.limit()
 - mi_cursor.sort()



MongoDB – Iniciando



○ Arreglos

- **var** valores = [1,2,3,4]
- **db.usuarios.insert**({user: "admin", permisos: valores})
- **db.usuarios.update**({}, { \$addToSet: { permisos: 4 } })
 - Lo agrega si no existe el elemento.
- **db.usuarios.update**({}, { \$push: { permisos: 4 } })
 - Lo agrega siempre.
- **db.usuarios.update**({}, { \$push: { permisos: { \$each: [5,6,7] } } })
- **db.usuarios.update**({}, { \$push: { permisos: { \$each: [8,9], \$position: 3 } } })
 - Insertar en una posición no funciona con "addToSet"
- **db.usuarios.update**({}, { \$push: { permisos: { \$each: [15,10], \$sort: 1 } } })
 - Inserta los elementos de forma ordenada

MongoDB – Iniciando



○ Arreglos

○ **db.usuarios.update**({}, { \$pull: { permisos: 3 } })

○ Elimina del arreglo el elemento 3

○ **db.usuarios.update**({}, { \$pull: { permisos: {\$gte: 5} } })

○ Elimina los elementos mayores o iguales a 5.

○ **db.usuarios.update**({}, { \$pullAll: { permisos: [4,5] } })

○ Elimina los valores indicados en el arreglo

MongoDB – Iniciando



○ Grupos

- **db.usuarios.aggregate**([{ \$group : { _id: "\$perfil" } }])

- "aggregate": calcula valores agregados para los datos en una colección.

- **db.usuarios.aggregate**([{ \$group : { _id: "\$perfil", "cantidad": { \$sum: 1 } } }])

- Agrupa por perfil y cuenta cantidad de valores.

- **db.productos.aggregate**([{ \$group : { _id: "\$categoria", "cantidad": { \$sum: 1 }, "suma_precio": { "sum: "\$precio" } } }])

- **db.productos.aggregate**([{ \$group : { _id: "\$categoria", "promedio": { \$avg: "\$precio" } } }])

MongoDB – Documento embebidos

○ Documentos embebidos

- Documentos que contienen otros documentos
- Similar a las relaciones en el modelo relacional

MongoDB – Documento embebidos

Persona

```
{ "_id": ObjectId(),  
  "dni": "un dni"  
  "apellido": "un apellido",  
  "nombre": "un nombre",  
  "localidad": {  
    "cod": "un código postal",  
    "nombre": "una localidad",  
    "prov": "una provincia"  
  }  
}
```

MongoDB – Documento embebidos

○ Ejemplo:

- `persona = {dni: "123456", apellido: "Ape 1", nombre: "Nom 1"}`
- `localidad = {cod: "7220", nombre: "Monte", prov: "Bs. As." }`
- `persona.localidad = localidad`
- `db.personas.insert(persona)`

MongoDB – Referenciar un documento

Persona

```
{ "_id": ObjectId(),  
  "dni": "un dni"  
  "apellido": "un apellido",  
  "nombre": "un nombre",  
  "id_loc": "un id localidad"  
}
```

Localidad

```
{ "_id": ObjectId(),  
  "cod": "un código postal",  
  "nombre": "una localidad",  
  "prov": "una provincia"  
}
```

MongoDB – Documento embebidos - Operaciones

- `db.personas.find({"nombre": "Nom 1"}).pretty()`
- `db.personas.find({"localidad.prov": "Buenos Aires"}).pretty()`
- `db.personas.find({"localidad.prov": "Buenos Aires"}, {"localidad": true}).pretty()`
 - Retorna solo los datos de la localidad.
- `db.personas.find({"localidad.prov": "Buenos Aires"}, {"localidad.nombre": true}).pretty()`
 - Retorna solo el campo nombre de la localidad.

MongoDB – Documento embebidos - Operaciones

- `db.personas.update({}, {$set: { "localidad.cod_postal": 1900 } })`
- `db.personas.update({}, {$set: { "localidad.cod_postal": 1900 } }, {multi: true})`
- `db.personas.remove( { "localidad.cod_postal": 1900 } )`

MongoDB – Documentos Embebidos – Relación uno a muchos

Persona

```
{ "_id": ObjectId(),  
  "dni": "un dni"  
  "apellido": "un apellido",  
  "nombre": "un nombre",  
  "localidad": {  
    "cod": "un código postal",  
    "nombre": "una localidad",  
    "prov": "una provincia"  
  }  
  "teléfono_1": { "pref": "un prefijo 1", "num": "un número 1" }  
  "teléfono_2": { "pref": "un prefijo 2", "num": "un número 2" }  
  "teléfono_3": { "pref": "un prefijo 3", "num": "un número 3" }  
}
```

MongoDB – Documentos Embebidos – Relación uno a muchos - Arreglos

Persona

```
{ "_id": ObjectId(),  
  "dni": "un dni"  
  "apellido": "un apellido",  
  "nombre": "un nombre",  
  "localidad": {  
    "cod": "un código postal",  
    "nombre": "una localidad",  
    "prov": "una provincia"  
  }  
  teléfonos: [  
    { "pref": "un prefijo 1", "num": "un número 1" },  
    { "pref": "un prefijo 2", "num": "un número 2" },  
    { "pref": "un prefijo 3", "num": "un número 3" }  
  ]  
}
```

MongoDB – Documentos Embebidos – Relación uno a muchos – Arreglos – Filtros

- `db.personas.find({ "telefonos.pref": "un prefijo" })`
- `db.personas.find({ telefonos: { $elemMatch: { pref: "un prefijo" } } })`
- `db.personas.find({ telefonos: { $elemMatch: { pref: "un prefijo", num: "un número" } } })`
- `db.personas.find({ telefonos: { $elemMatch: { pref: "un prefijo" } } }, { _id: false, telefonos: true, dni: true })`
- `db.personas.find({ telefonos: { $elemMatch: { pref: "un prefijo" } } }, { _id: false, telefonos: { $elemMatch: { pref: "otro prefijo" } }, dni: true })`
 - En este caso indicamos el sub documento con el que se quiere trabajar.
- `db.personas.find( { "telefonos.pref": un prefijo }, { _id: false, "telefonos.$": true } )`

MongoDB – Documentos Embebidos – Relación uno a muchos – Arreglos – Actualización

- `db.personas.update({ "telefonos.pref": "un prefijo"}, { $set: { "apellido": "otro apellido" } } )`
- `db.personas.update( { "telefonos.pref": "un prefijo"}, { $set: { "telefonos.$": { pref: "otro prefijo", num: " un numero" } } } )`
- `db.personas.update( { "telefonos.pref": "un prefijo"}, { $set: { "telefonos.$.num": "un numero" } } )`



GRACIAS

mongoDB